



Image (c) Ryan Somma [www.flickr.com/people/ideonexus]
Reused under the terms of CC-BY-SA 2.0 license

Extending Python via Shared Libraries

Python is one of the most popular programming languages ever—its great productivity, flexibility and general-purpose nature efficiently address areas ranging from Internet applications to system uses. Much of Python's power comes from extensions written in languages like C and C++. In this article, we look at extending Python with Ctypes, SWIG, Pyrex and Cython, and the pros and cons of each.

The ability to call C/C++ functions from Python code, through extensions, exposes lower-level 'system' functionality that would otherwise not be available in Python. Also, you can optimise the performance-critical parts of your applications by creating those as C/C++ extension modules, which will be compiled to native code, yielding better performance than the interpreted Python byte-code.

How can we extend Python?

The standard and most widely used implementation of Python is CPython, which is programmed in C. CPython has a C API to create extensions, but this approach of extending Python is complicated, tedious, and error-prone. You have to manually code everything from data conversion to garbage collection. Leaving even a small detail unattended increases the likelihood of crashes.